

Python-Tuple 25 MCQ Questions

Question - 1

Statement (S): A tuple is immutable, but objects stored inside a tuple need not be immutable.

Reason (R): Tuple immutability prevents modification of the tuple's references, but does not recursively make referenced objects immutable.

A. Both S and R are true, and R correctly explains S

B. Both S and R are true, but R does not correctly explain S

C. S is true, but R is false

D. S is false, but R is true

Question - 2

Statement (S): The following code raises **TypeError**.

```
t = ([1, 2],)
```

```
t[0] += [3]
```

Reason (R): += first mutates the list and then attempts to assign the resulting object back to t[0].

A. Both S and R are true, and R correctly explains S

B. Both S and R are true, but R does not correctly explain S

C. S is true, but R is false

D. S is false, but R is true

Question-3

Statement (S): The following two tuples are equal.

```
a = (1, 2, [3])
```

```
b = (1, 2, [3])
```

Reason (R): Tuple equality compares corresponding elements using their equality operation rather than requiring the elements to be the same objects.

A. Both S and R are true, and R correctly explains S

B. Both S and R are true, but R does not correctly explain S

C. S is true, but R is false

D. S is false, but R is true

Question-4

Statement (S): The following tuple cannot be used as a dictionary key.

```
t = (1, 2, [3, 4])
```

Reason (R): A tuple is hashable only when all of its elements are hashable.

A. Both S and R are true, and R correctly explains S

B. Both S and R are true, but R does not correctly explain S

C. S is true, but R is false

D. S is false, but R is true

Question-5

Statement (S): In the following code, t still contains [1, 2].

```
x = [1, 2]
```

```
t = (x,)
```

```
x = x + [3]
```

Reason (R): x + [3] creates a new list and rebinds x; it does not mutate the original list referenced by the tuple.

A. Both S and R are true, and R correctly explains S

B. Both S and R are true, but R does not correctly explain S

C. S is true, but R is false

D. S is false, but R is true

Question-6

Statement (S): Tuple unpacking requires the number of variables to exactly match the number of elements unless starred unpacking is used.

Reason (R): A starred target can absorb zero or more remaining elements during unpacking.

A. Both S and R are true, and R correctly explains S

B. Both S and R are true, but R does not correctly explain S

C. S is true, but R is false

D. S is false, but R is true

Question-7

Statement (S): The following code produces a TypeError.

```
t = (1, 2)
```

```
print(t < (1, "2"))
```

Reason (R): Python performs tuple comparison lexicographically and eventually attempts to compare 2 and "2".

A. Both S and R are true, and R correctly explains S

B. Both S and R are true, but R does not correctly explain S

C. S is true, but R is false

D. S is false, but R is true

Question-8

Statement (S): A tuple containing a mutable list can be equal to another tuple containing a different list object.

```
a = ([1, 2],)
```

```
b = ([1, 2],)
```

Reason (R): Tuple equality depends on element equality (==), not necessarily object identity (is).

A. Both S and R are true, and R correctly explains S

B. Both S and R are true, but R does not correctly explain S

C. S is true, but R is false

D. S is false, but R is true

Question-9

Statement (S): The following operation can modify an object inside a tuple even though the tuple itself is immutable.

```
t = ({1, 2},)
```

```
t[0].add(3)
```

Reason (R): Immutability of a tuple does not imply immutability of objects referenced by its elements.

A. Both S and R are true, and R correctly explains S

B. Both S and R are true, but R does not correctly explain S

C. S is true, but R is false

D. S is false, but R is true

Question-10

Statement (S): The following two expressions can produce different results even though both appear to perform "addition" on the list inside a tuple.

```
t = ([1, 2],)
```

```
t[0] += [3]
```

and

```
t = ([1, 2],)
```

```
t = (t[0] + [3],)
```

Reason (R): += may mutate the existing list before the tuple-item assignment is attempted, whereas + creates a new list

.

A. Both S and R are true, and R correctly explains S

B. Both S and R are true, but R does not correctly explain S

C. S is true, but R is false

D. S is false, but R is true

Question-11

```
1 t = (1, 2, 3)
2 s = t[:]
3 print(t is s, t == s)
```

A) True True

B) True False

C) False True

D) False False

Question-12

```
1 print((1, 'a') > (1, 2))
```

A) True

B) False

C) TypeError

D) ValueError

Question-13

```
1 t = (0, 1, 2)
2 print(False in t, True in t)
```

A) False False

B) False True

C) True True

D) True False

Question-14

```
1 t = ([1],) * 3
2 t[0].append(2)
3 print(t[1])
```

A) [1]

B) [1, 2]

C) ([1, 2], [1, 2], [1, 2])

D) Raises IndexError

Question-15

```
1 t = (1, 2)
2 u = (*t, 3, *t)
3 print(u)
```

A) (1, 2, 3, 1, 2)

B) ((1, 2), 3, (1, 2))

C) SyntaxError

D) (1, 2, 3)

Question-16

```
1 a, *b, *c, d = (1, 2, 3, 4, 5)
```

- A) a=1, b=[2,3], c=[4], d=5
- B) SyntaxError
- C) ValueError (too many values to unpack)
- D) a=1, b=[2], c=[3,4], d=5

Question-17

```
1 t1 = (1000, 2000)
2 t2 = (1000, 2000)
3 print(t1 is t2, t1[0] is t2[0])
```

- A) False False
- B) True True
- C) False True
- D) True False

Question-18

```
1 t = (1, 2, (1, 2), 1, 2)
2 print(t.count(1), t.index((1, 2)))
```

- A) 2, 2
- B) 2, 1
- C) 1, 2
- D) 1, 1

Question -19

```
1 t = (1, 2)
2 old_id = id(t)
3 t += (3, 4)
4 print(id(t) == old_id)
```

- A) True
- B) False
- C) Depends on Python interning
- D) TypeError

Question-20

```
1 from collections import namedtuple
2
3 Point = namedtuple('Point', ['x', 'y', 'z'], defaults=[10, 20])
4 p1 = Point(5)
5 p2 = Point(5, 6)
6
7 print(p1.z, p2.z)
```

- A) 10 10
- B) 20 20
- C) 10 20
- D) 20 10

Question-21

```
1 from collections import namedtuple
2
3 Point = namedtuple('Point', ['x', 'y'])
4 data = {'x': 100, 'y': 200}
5
6 p1 = Point._make(data)
7 p2 = Point(**data)
8
9 print(p1, p2)
```

- A) Point(x=100, y=200) Point(x=100, y=200)
- B) Point(x='x', y='y') Point(x=100, y=200)
- C) TypeError Point(x=100, y=200)
- D) Point(x=100, y=200) TypeError

Question-22

```
1 from collections import namedtuple
2
3 T1 = namedtuple('T1', ['a', 'b', 'c'], defaults=[1, 2])
4 T2 = namedtuple('T2', ['a', 'b', 'c'], defaults=(3,))
5
6 print(T1._field_defaults)
7 print(T2._field_defaults)
```

- A) {'b': 1, 'c': 2} and {'c': 3}
- B) {'a': 1, 'b': 2} and {'a': 3}
- C) {'c': 1, 'b': 2} and {'c': 3}
- D) {'b': 1, 'c': 2} and {'a': 3}

Question-23

Which statement is TRUE?

- A. If a tuple contains a mutable object, the tuple itself becomes mutable.
 - B. A tuple guarantees that the complete object graph reachable through its elements is immutable.
 - C. A tuple prevents modification of its own element references, but it does not necessarily prevent mutation of objects referenced by those elements.
 - D. Tuple immutability means every operation involving an element of the tuple is prohibited.
-

Question-24

Which statement is MOST CORRECT regarding tuple hashability?

- A. Every tuple is hashable because every tuple is immutable.
 - B. A tuple is hashable only if all of its elements are hashable.
 - C. A tuple is hashable only if all of its elements are immutable.
 - D. Tuple hashability depends only on the number of elements it contains.
-

Question-25

Why can a tuple potentially be used as a dictionary key while a list cannot?

- A. Tuples occupy less memory than lists.
- B. Tuples maintain insertion order while lists do not.
- C. Dictionary keys must be hashable, and tuples can be hashable whereas lists are unhashable.
- D. Python internally converts every tuple into a string before using it as a key.